Biomedical Knowledge Graph using Neo4j

Alternative Approach to PrimeKG

Name: Aatazaz Javed
Submitted To: Dr. Muhammad Ishaq
Course: Bioinformatics
Examination: Mid-Term
Program: MS Data Science 2nd Semester
Institution: Institute of Management Sciences Peshawar

This project presents the development of a biomedical knowledge graph using Python and Neo4j. The system integrates biomedical entities such as drugs, diseases, and genes into a connected graph structure. Unlike large-scale systems such as PrimeKG, this project adopts a simplified and reproducible approach using CSV datasets and automated Python scripts. The resulting knowledge graph demonstrates key features including entity relationships, graph visualization, and validation.

---

1. Introduction

Biomedical knowledge graphs are used to represent relationships between biological entities. These graphs are widely used in:

- Drug discovery
- Disease analysis
- Precision medicine

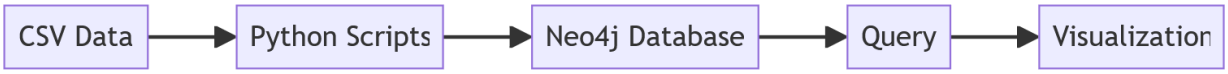
Large datasets like PrimeKG are complex. Therefore, this project develops a **lightweight alternative** using Neo4j.

---

2. Methodology

System Pipeline

CSV Data → Python Scripts → Neo4j Database → Query → Visualization



Tools Used

- Neo4j (Graph Database)
- Python (Automation)
- Pandas (Data handling)

3. Dataset Description

The dataset consists of:

Entity	Count
Drugs	20
Diseases	20
Genes	20

Total Nodes: 60

Total Relationships: 40

---

4. Node Types

- Drug
- Disease
- Gene

Each node contains:

- ID
- Name

---

5. Relationships

Relationship	Description
TREATS	Drug → Disease
ASSOCIATED_WITH	Gene → Disease

---

6. Implementation

The system was implemented using Python scripts:

- CSV files loaded using pandas
- Neo4j driver used for connection
- Cypher queries executed automatically

7. Neo4j Desktop

Neo4j is a leading graph database management system designed to store and manage highly connected data. Unlike traditional relational databases that organize data in tables, Neo4j uses a graph-based model consisting of **nodes**, **relationships**, and **properties**. This structure allows efficient representation of complex relationships, making it particularly suitable for domains such as social networks, recommendation systems, and biomedical informatics.

In Neo4j, **nodes** represent entities (e.g., drugs, diseases, genes), while **relationships** define how these entities are connected (e.g., a drug *treats* a disease). Each node and relationship can also have attributes, known as properties, which provide additional information. Neo4j uses a powerful query language called **Cypher**, which enables intuitive querying and manipulation of graph data.

7.1 Use of Neo4j in This Project

In this project, Neo4j is used as the core database system to construct and manage a biomedical knowledge graph. The graph represents three primary types of entities: **Drugs, Diseases, and Genes**. These entities are modeled as nodes, and their interactions are represented through relationships such as:

- **TREATS** (Drug → Disease)
- **ASSOCIATED_WITH** (Gene → Disease)

The dataset, initially stored in CSV format, is imported into Neo4j using Python scripts. The Neo4j Python driver is used to establish a connection and execute Cypher queries programmatically. This allows automated creation of nodes and relationships without manual intervention.

Once the data is loaded, Neo4j provides powerful visualization capabilities through its browser interface, enabling users to explore the graph structure interactively. Additionally, Cypher

queries are used to validate the graph, calculate node and relationship counts, and perform basic analysis such as identifying the most connected disease.

Overall, Neo4j serves as the backbone of this project by enabling efficient storage, querying, and visualization of biomedical relationships, thereby demonstrating the practical implementation of a knowledge graph system.

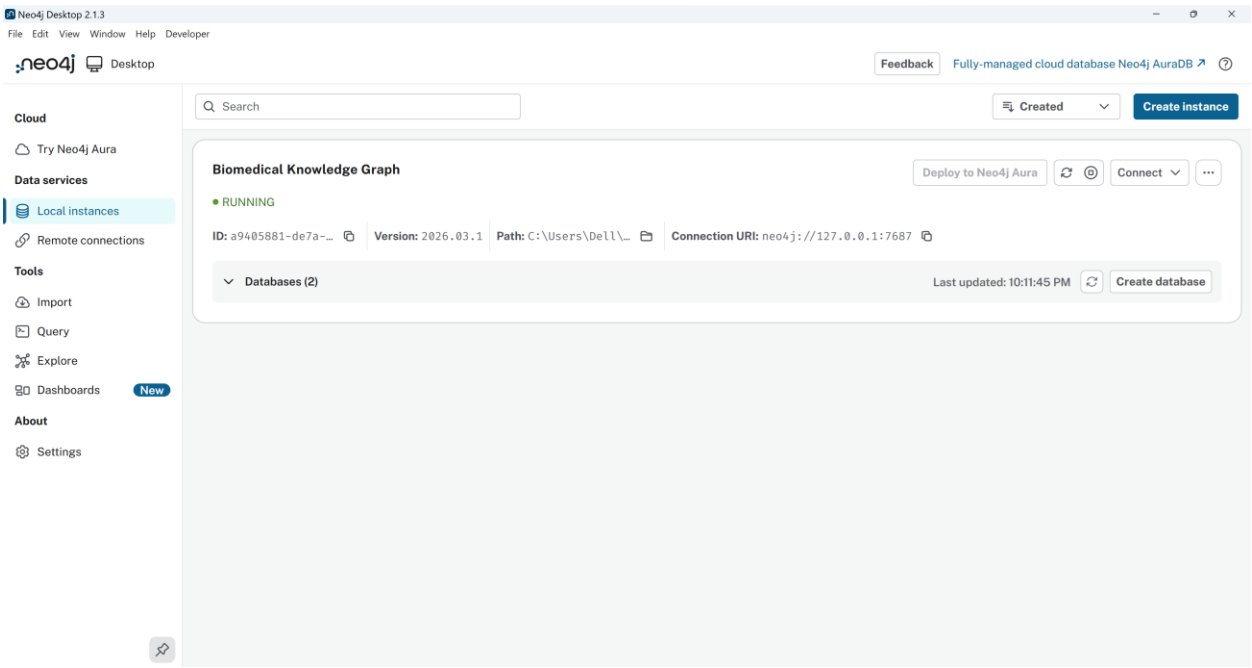
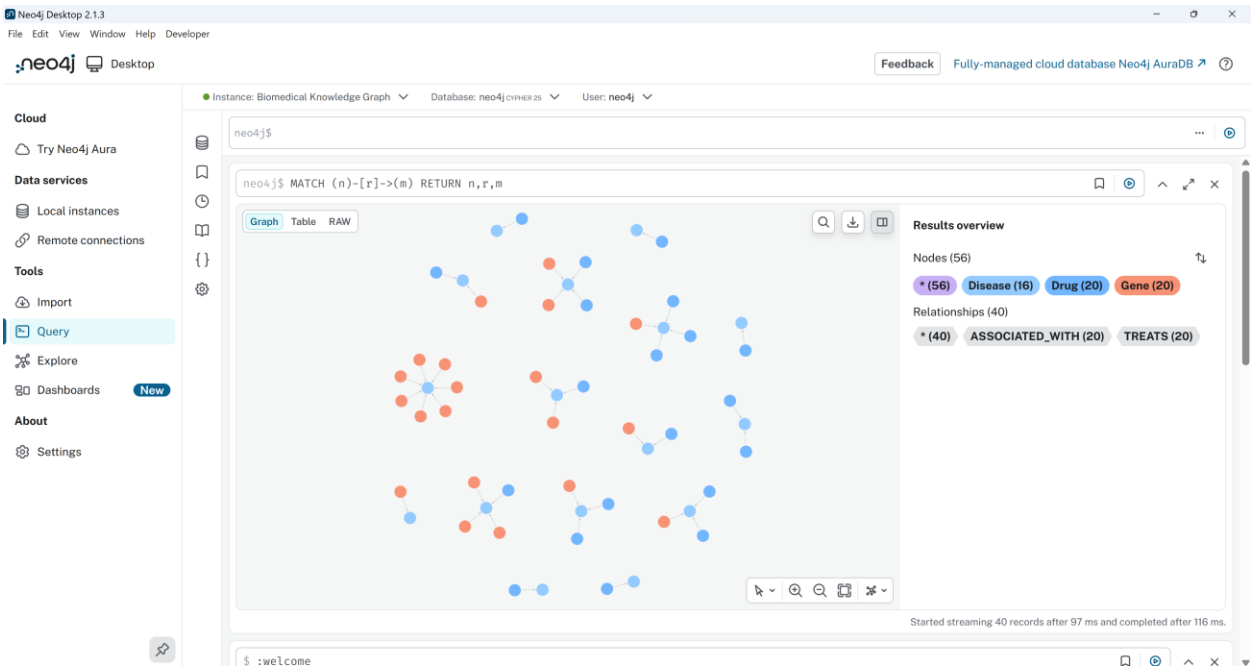


Figure 1: Neo4j Database Running



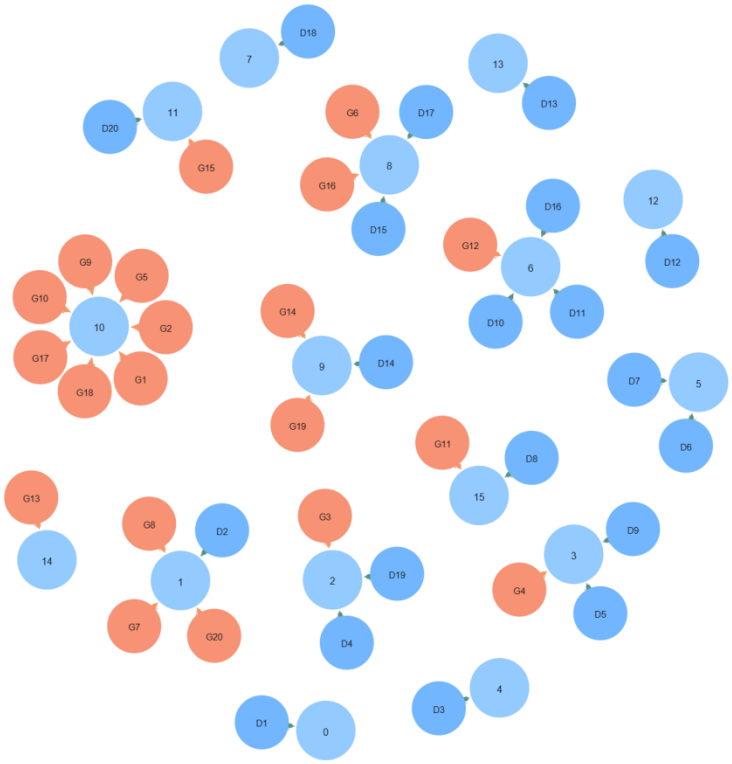


Figure 2: Graph Visualization

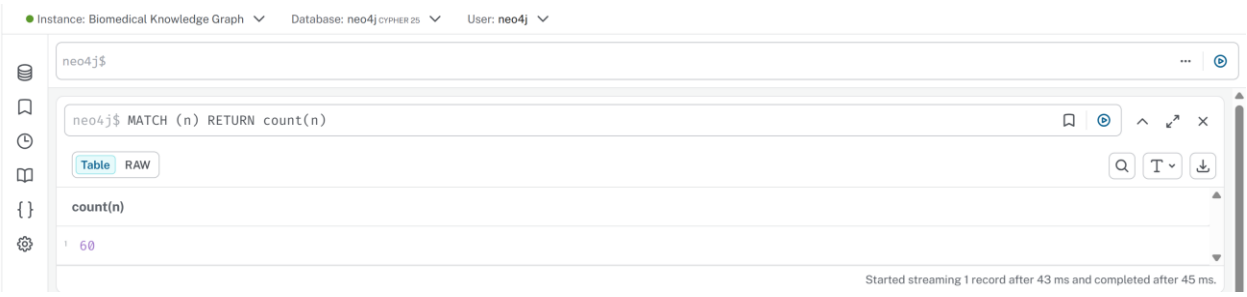


Figure 3: Query Execution

8. Validation Results

Metric	Value
Total Nodes	60
Total Relationships	40
Drug Nodes	20
Disease Nodes	20d
Gene Nodes	20

Hub Analysis

Most connected disease: **Inflammation**

9. Discussion

The project demonstrates:

- Efficient graph modeling
- Automated data integration
- Scalable architecture
- Query-based validation

Compared to PrimeKG, this approach is:

- Simpler
- Reproducible
- Suitable for academic use

The developed biomedical knowledge graph demonstrates the effectiveness of graph-based data modeling for representing complex biological relationships. By integrating drugs, diseases, and genes into a unified graph structure, the project highlights how interconnected biomedical data can be efficiently stored, queried, and analyzed using **Neo4j Desktop**.

One of the key strengths of this approach is its ability to model many-to-many relationships naturally. Unlike traditional relational databases, where such relationships require complex joins, Neo4j allows direct connections between entities, resulting in improved query performance and simpler data representation. This becomes particularly valuable in biomedical domains, where entities are highly interdependent and relationships are critical for understanding biological processes.

The use of Python automation further enhances the system by enabling scalable and reproducible data integration. Instead of manually inserting data, the project employs scripts to read structured CSV datasets and convert them into graph nodes and relationships using Cypher queries. This pipeline ensures consistency, reduces human error, and allows easy expansion of the dataset.

From an analytical perspective, the graph supports basic insights such as node counts, relationship distribution, and hub detection. For example, identifying the most connected disease (Inflammation) demonstrates how graph centrality can be used to detect important biomedical entities. Although the current analysis is simple, the framework can be extended to include advanced techniques such as centrality measures, clustering, and path analysis.

However, the project also has certain limitations. The dataset used is synthetic and relatively small compared to real-world biomedical knowledge graphs like PrimeKG. Additionally, the relationships are limited to only two types (TREATS and ASSOCIATED_WITH), whereas real biomedical systems include a wider range of interactions such as drug-drug interactions, protein pathways, and side effects.

Despite these limitations, the project successfully establishes a strong foundation for biomedical knowledge graph construction. It demonstrates how a simplified and lightweight system can

replicate the core principles of large-scale knowledge graphs while remaining accessible and easy to implement in an academic setting.

---

10. Inference

This project successfully presents the design and implementation of a biomedical knowledge graph using Neo4j and Python as a simplified alternative to large-scale systems such as PrimeKG. By integrating drugs, diseases, and genes into a unified graph structure, the project demonstrates the practical application of graph databases in biomedical informatics.

The use of **Neo4j Desktop** enables efficient storage and querying of interconnected data, while Python-based automation ensures scalability and reproducibility of the data pipeline. The system effectively transforms structured CSV datasets into a fully functional graph database, allowing users to visualize relationships and perform validation through Cypher queries.

The results confirm that even a moderately sized dataset can provide meaningful insights when modeled as a graph. The ability to identify key entities and relationships highlights the potential of knowledge graphs in supporting data-driven biomedical research.

In conclusion, this project not only fulfills the objective of building a biomedical knowledge graph but also demonstrates the advantages of graph-based approaches over traditional data models. The framework developed in this study can be further extended by incorporating real-world biomedical datasets, additional entity types, and advanced analytical methods. Such enhancements would enable more comprehensive exploration of biological systems and contribute to applications in drug discovery, disease prediction, and precision medicine.